

CSE331 - Assignment #1: Sorting Algorithm Analysis

Seong-Ji Yang (20211181)

UNIST

South Korea

yangsj@unist.ac.kr

1 PROBLEM STATEMENT

Sorting, the fundamental process of arranging items into a defined order, underpins countless computational tasks, from database indexing to graphical rendering. While conceptually simple, achieving efficient sorting is critical. A naive brute-force approach, examining all $n!$ possible permutations, suffers from factorial time complexity, rendering it utterly impractical for non-trivial input sizes n .

The efficiency limits of comparison-based sorting are well-understood through the decision tree model. Any such algorithm can be represented as a binary tree where internal nodes denote comparisons ($a_i : a_j$) and each leaf corresponds to a unique sorted permutation of the input. Since there are $n!$ possible permutations, the tree must have at least $n!$ leaves. A binary tree of height h has at most 2^h leaves. Therefore, we must have $n! \leq 2^h$. Taking logarithms gives the lower bound on the number of comparisons required in the worst case:

$$h \geq \log(n!) = \sum_{i=1}^n \log i$$

A simple lower bound can be derived by considering only the larger half of the terms in the summation:

$$\log(n!) \geq \sum_{i=n/2+1}^n \log i \geq \sum_{i=n/2+1}^n \log\left(\frac{n}{2}\right) = \frac{n}{2} \log\left(\frac{n}{2}\right) = \Omega(n \log n).$$

This $\Omega(n \log n)$ lower bound establishes a theoretical benchmark for comparison sorts[5], starkly contrasting with the $O(n!)$ complexity of brute force and motivating the development of algorithms like Merge Sort and Heap Sort that achieve this optimal asymptotic bound.

This project undertakes the implementation and comparative analysis of 12 distinct sorting algorithms, categorized as:

- (1) Six conventional comparison-based algorithms: Merge Sort, Heap Sort, Bubble Sort, Insertion Sort, Selection Sort, and Quick Sort. These foundational algorithms illustrate core sorting techniques.
- (2) Six contemporary algorithms: Library Sort, Tim Sort, Cocktail Shaker Sort, Comb Sort, Tournament Sort, and Introsort. These often refine or hybridize basic techniques for improved practical performance or robustness.

For each algorithm, the analysis will encompass its underlying design rationale (including implementation choices), a detailed theoretical time complexity analysis (including derivations for best, average, and worst cases where applicable), space complexity, and **stability**. Furthermore, empirical evaluations will be conducted by measuring execution time and memory usage across diverse input datasets (ascending, descending/reverse sorted, random, partially sorted) with sizes ranging from 1K to 1M elements, averaging results over a minimum of 10 runs per test configuration.

2 BASIC SORTING ALGORITHMS

This section provides a detailed analysis of six fundamental comparison-based sorting algorithms, including their design principles, properties, and time complexity derivations, highlighting implementation choices where relevant.

2.1 Merge Sort

Characteristics: Stable, Not In-place.

Design Rationale (Implementation): Implemented using the standard recursive divide-and-conquer approach. The core idea is to recursively sort the left and right halves of the array and then merge the two sorted halves. The Merge function in this implementation utilizes temporary arrays L and R to hold the elements of the two halves, comparing elements and placing the smaller one back into the original array segment. Sentinels simplify the merging loop. This design prioritizes clarity and guarantees stability. The use of $O(n)$ auxiliary space for L and R is consistent with the standard rationale for Merge Sort[5].

2.1.1 Time Complexity Analysis Derivation.

Recurrence Relation:

- **Divide:** Splitting the array takes $O(1)$ time.
- **Conquer:** Two recursive calls on subproblems of size $n/2$ take $2T(n/2)$.
- **Combine:** The Merge procedure iterates through n elements, performing constant work per element. Copying to/from temporary arrays also takes $O(n)$. Total merge cost is $O(n)$.
- **Recurrence:** $T(n) = 2T(n/2) + O(n)$, with $T(1) = O(1)$.

Solving the Recurrence (Master Method): Parameters $a = 2, b = 2, f(n) = O(n)$. Compare $f(n)$ with $n^{\log_b a} = n^{\log_2 2} = n$. Case 2 applies ($f(n) = O(n^{\log_b a} \log^0 n)$), yielding $T(n) = O(n \log n)$.

All Cases: The work performed depends only on the size n . Thus, the complexity is consistently $O(n \log n)$ for Best, Average, and Worst cases.

In Conclusion: Merge Sort is **stable** but **not in-place** ($O(n)$ space). Advantage: Guaranteed $O(n \log n)$ performance and stability. Disadvantage: Higher space complexity.

2.2 Heap Sort

Characteristics: Unstable, In-place.

Design Rationale (Implementation): Implemented in-place using a binary max-heap structure implicitly within the array. 1. 'Build Max-Heap': Converts the input array into a max-heap in $O(n)$ time using a bottom-up approach, calling `Heapify` from the last non-leaf node down to the root. 2. 'Sort Phase': Repeatedly swaps the root

(maximum element) with the last element of the current heap section, decreases heap size, and calls `Heapify` on the root ($O(\log n)$ time) to restore the heap property. This overwrites the sorted elements at the end. This implementation adheres to the standard Heap Sort rationale for in-place sorting with $O(1)$ auxiliary space[5].

2.2.1 Time Complexity Analysis Derivation.

Phase Costs:

- ‘Build Max-Heap’: Takes $O(n)$ time.
- Sort Phase: $n - 1$ iterations. Each involves $O(1)$ swap + $O(\log k)$ `Heapify`. Total cost dominated by $\sum_{k=1}^{n-1} O(\log k) = O(n \log n)$.

All Cases: Dominated by the sort phase, yielding $O(n \log n)$ for Best, Average, and Worst cases.

In Conclusion: Heap Sort is **unstable** but **in-place**. Advantage: Guaranteed $O(n \log n)$ time with $O(1)$ space. Disadvantage: Unstable, potentially poor cache performance.

2.3 Bubble Sort

Characteristics: Stable, In-place.

Design Rationale (Implementation): Uses nested loops. The inner loop compares adjacent elements and swaps them if out of order, "bubbling" the largest element to the end[5]. The outer loop repeats this process. This specific implementation intentionally omits the common early-exit optimization (stopping if a pass causes no swaps), differing from optimized versions to serve as a clear $O(n^2)$ baseline as requested for analysis.

2.3.1 Time Complexity Analysis Derivation (Brute-force).

Summation Analysis: The number of comparisons in the nested loops is always $\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 1 = \frac{(n-1)n}{2}$. This arithmetic series sum is $O(n^2)$.

Case Analysis: Without optimization, $O(n^2)$ comparisons are always made. Swaps vary ($O(1)$ best, $O(n^2)$ worst/average). Since comparisons dominate, the time complexity is $O(n^2)$ for Best, Average, and Worst cases.

In Conclusion: **Stable, in-place**. Advantage: Extreme simplicity. Disadvantage: Highly inefficient ($O(n^2)$) due to the lack of optimization in this version.

2.4 Insertion Sort

Characteristics: Stable, In-place.

Design Rationale (Implementation): Builds the sorted array one element at a time. It iterates from the second element ($i=2$ to n). For each element $key = A[i]$, it finds the correct position j in the sorted subarray $A[1..i-1]$ by shifting larger elements ($A[j]$) right until the spot is found, then inserts key . This implementation follows the standard iterative rationale[5].

2.4.1 Time Complexity Analysis Derivation (Brute-force).

Summation Analysis: In the worst case (reverse sorted), inserting the i -th element takes $i - 1$ comparisons/shifts. The total time is

proportional to $\sum_{i=2}^n O(i-1) = O(1+2+\dots+(n-1)) = O\left(\frac{(n-1)n}{2}\right)$, which is $O(n^2)$. The average case also leads to a $O(n^2)$ sum.

Case Analysis: Worst/Average $O(n^2)$. Best case (already sorted) involves $\sum_{i=2}^n O(1) = O(n)$ comparisons and no shifts, resulting in $O(n)$ time.

In Conclusion: **Stable, in-place**. Advantage: Simple, adaptive ($O(n)$ best case), efficient for small N . Disadvantage: $O(n^2)$ for large random data.

2.5 Selection Sort

Characteristics: Unstable, In-place.

Design Rationale (Implementation): Divides the list into sorted (left) and unsorted (right) portions. In each pass i , finds the minimum in the unsorted portion $A[i..n]$ and swaps it with $A[i]$. The rationale is minimizing swaps. This implementation follows the standard approach[5].

2.5.1 Time Complexity Analysis Derivation (Brute-force).

Operation Counts: Finding the minimum in pass i requires $n - i$ comparisons. Total comparisons: $\sum_{i=1}^{n-1} (n - i) = \frac{(n-1)n}{2}$, which is $O(n^2)$. Swaps: Always $n - 1$, i.e., $O(n)$.

All Cases: The $O(n^2)$ comparisons dominate the $O(n)$ swaps. Therefore, the time complexity is $O(n^2)$ for Best, Average, and Worst cases.

In Conclusion: **Unstable, in-place**. Advantage: Minimal data movement (swaps). Disadvantage: Consistently slow $O(n^2)$ time.

2.6 Quick Sort

Characteristics: Unstable, Not strictly In-place.

Design Rationale (Implementation): Employs divide-and-conquer[4]. Selects a pivot (here, always $V[start]$) and partitions elements around it. Sub-arrays are sorted recursively. This implementation deviates for large N ($> 10k$) by using an iterative approach with a queue to manage subproblems. This avoids stack overflow from deep recursion (e.g., on sorted data with this pivot strategy).

2.6.1 Time Complexity Analysis Derivation.

Recurrence Relation: $T(n) = T(k) + T(n - 1 - k) + O(n)$, where $O(n)$ is partitioning cost.

Case Analysis: Worst case ($k = 0$) leads to $T(n) = T(n-1) + O(n)$, solving to $O(n^2)$. Best case ($k \approx n/2$) yields $T(n) = 2T(n/2) + O(n)$, resulting in $O(n \log n)$. Average case also yields $O(n \log n)$. Space: $O(\log n)$ average stack/queue, $O(n)$ worst-case stack/queue.

In Conclusion: **Unstable, not strictly in-place**. Advantage: Excellent $O(n \log n)$ average time. Disadvantage: $O(n^2)$ worst case from simple pivot choice[5].

3 ADVANCED SORTING ALGORITHMS

This section analyzes six contemporary sorting algorithms, focusing on their design rationale based on the C++ implementation choices, complexity, and properties. Pseudocode is based on the provided C++ files.

Phase Costs: Run creation $O(n)$. Balanced merging $O(n \log n)$ worst-case.

Cases: Worst: $O(n \log n)$. Average: $O(n \log n)$. Best (few long runs): $O(n)$.

In Conclusion: Tim Sort: **Stable**, adaptive, **not strictly in-place**. Advantage: Excellent real-world performance ($O(n)$ best), stable. Disadvantage: Complex to implement.

3.3 Cocktail Shaker Sort

Characteristics: Stable, In-place.

Design Rationale (Implementation): A bidirectional Bubble Sort[5]. Passes L→R bubbling large elements up, then R→L bubbling small elements down. This C++ implementation intentionally omits early-exit optimizations (tracking bounds/swaps) and uses fixed outer loop bounds (for $i=1$ to $(1+\text{check_size})/2$), differing from optimized versions to force $O(n^2)$ for baseline comparison.

Algorithm 3: Cocktail Shaker Sort (based on C++ implementation cocktail_shaker_sort.cpp)

Data: Array V (1-based index)

Result: Sorted array V

Procedure CocktailShakerSortV:

```

 $n \leftarrow \text{size}(V) - 1;$ 
for  $i \leftarrow 1$  to  $(1 + n)/2$  do
    for  $j \leftarrow i$  to  $n - i + 1$  do
        if  $V[j] > V[j + 1]$  then
            swap  $V[j]$  and  $V[j + 1]$ 
        end
    end
    for  $j \leftarrow n - i$  down to  $i + 1$  do
        if  $V[j - 1] > V[j]$  then
            swap  $V[j - 1]$  and  $V[j]$ 
        end
    end
end

```

3.3.1 Time Complexity Analysis Derivation (Brute-force).

Summation Analysis: Each outer iteration i performs $\approx (n - 2i) + (n - 2i) = O(n - i)$ comparisons. Total comparisons $\sum_{i=1}^{n/2} O(n - i) = O(n^2)$.

Case Analysis: Fixed passes mean $O(n^2)$ comparisons always. Time complexity is consistently $O(n^2)$ (Best, Average, Worst).

In Conclusion: Cocktail Shaker Sort (C++ version): **Stable, in-place**. Advantage: Simple concept. Disadvantage: Inefficient implementation ($O(n^2)$) lacking optimization.

3.4 Comb Sort

Characteristics: Unstable, In-place.

Design Rationale (Implementation): Improves Bubble Sort using a shrinking gap (inc, starts large, shrinks by factor $3/4$), a concept related to variations explored earlier[3]. Compares elements inc apart. Finishes with an optimized Bubble Sort phase (gap=1, early exit via last_swap_pos). This C++ code implements this standard Comb Sort logic.

Algorithm 4: Comb Sort (based on C++ implementation comb_sort.cpp)

Data: Array V (1-based index)

Result: Sorted array V

Procedure CombSortV:

```

 $n \leftarrow \text{size}(V) - 1;$   $\text{inc} \leftarrow n/2;$ 
while  $\text{inc} > 0$  do
    for  $j \leftarrow 1$  to  $n - \text{inc}$  do
        if  $V[j] > V[j + \text{inc}]$  then
            swap  $V[j]$  and  $V[j + \text{inc}]$ 
        end
    end
     $\text{inc} \leftarrow \lfloor 3 \times \text{inc} / 4 \rfloor;$ 
end
 $\text{limit} \leftarrow n;$  while  $\text{limit} > 1$  do
     $\text{last\_swap\_pos} \leftarrow 0;$ 
    for  $i \leftarrow 1$  to  $\text{limit} - 1$  do
        if  $V[i] > V[i + 1]$  then
            swap  $V[i]$  and  $V[i + 1];$ 
             $\text{last\_swap\_pos} \leftarrow i;$ 
        end
    end
    if  $\text{last\_swap\_pos} == 0$  then
        break
    end
     $\text{limit} \leftarrow \text{last\_swap\_pos};$ 
end

```

3.4.1 Time Complexity Analysis Derivation.

Phase Costs: Gap phase average performance near $O(n \log n)$ with shrink factor ≈ 1.3 , worst $O(n^2)$. Final bubble phase adaptive.

Cases: Worst: $O(n^2)$. Average: Near $O(n \log n)$. Best: $O(n \log n)$.

In Conclusion: Comb Sort (C++ version): **Unstable, in-place**. Advantage: Simple improvement, good average performance. Disadvantage: $O(n^2)$ worst case, unstable.

3.5 Tournament Sort

Characteristics: Unstable, Not In-place.

Design Rationale (Implementation): Uses an explicit binary tree (v storing indices) simulating a tournament, often implemented using an array similar to a heap[8][5]. Rationale: Find min in $O(\log n)$ after $O(n)$ setup. Builds tree ($O(n)$), then repeatedly extracts root

(min index), records value (tmp), replaces leaf with infinity, updates path (UpdateTreePath, $O(\log n)$). This C++ implementation uses an array for the tree.

Algorithm 5: Tournament Sort (based on C++ implementation tournament_sort.cpp)

Data: Array V_{data} (1-based index)
Result: Sorted array V_{data}
Procedure TournamentSort(V_{data}):

```

     $n \leftarrow \text{size}(V_{data}) - 1$ ;
     $\text{tree\_size} \leftarrow 1$ ; while  $\text{tree\_size} < n$  do
        |  $\text{tree\_size} \leftarrow \text{tree\_size} \times 2$ 
    end
    ;
    Init vector  $v$  (size  $\text{tree\_size} + n$ ); Copy  $V_{data}$  to leaf
    section;
    for  $i \leftarrow \text{tree\_size} - 1$  down to 1 do
        |  $\text{child1\_loc} \leftarrow \dots$ ;  $\text{child2\_loc} \leftarrow \dots$ ;  $v[i] \leftarrow$ 
        |  $\text{index\_of\_min}(v[\text{child1\_loc}], v[\text{child2\_loc}])$ ;
    end
    ;
    Init  $\text{tmp}[1..n + 1]$ ;
    for  $i \leftarrow 1$  to  $n$  do
        |  $\text{winner\_loc} \leftarrow v[1]$ ;
        |  $\text{tmp}[i] \leftarrow v[\text{winner\_loc}]$ ;
        |  $v[\text{winner\_loc}] \leftarrow \text{INT\_MAX}$ ;
        | UpdateTreePath ( $\text{winner\_loc}/2$ );
    end
     $V_{data} \leftarrow \text{tmp}$ ;
```

3.5.1 Time Complexity Analysis Derivation.

Phase Costs: Building tree $O(n)$. n extractions, each $O(\log n)$ path update. Total extraction $O(n \log n)$.

Cases: Consistently $O(n \log n)$ (Best, Average, Worst).

In Conclusion: Tournament Sort (C++ version): **Unstable, not in-place** ($O(n)$ space). Advantage: Guaranteed $O(n \log n)$ time. Disadvantage: High space usage.

3.6 Introsort

Characteristics: Unstable, Not In-place (due to stack).

Design Rationale (Implementation): Hybrid: Quick Sort for average speed, Heap Sort for worst-case guarantee, as proposed by Musser[6]. Rationale: Use Quick Sort, but switch to Heap Sort if recursion depth exceeds $\lfloor \log_2(\text{size}(V) - 1) \rfloor$. Use Insertion Sort for small partitions (< 16). This C++ implementation (IntrosortRecursive) follows this but uses a simple first-element pivot for Quick Sort.

3.6.1 Time Complexity Analysis Derivation.

Phase Costs: Quick Sort phases dominate average performance $O(n \log n)$. Heap Sort fallback ensures $O(n \log n)$ worst-case. Insertion Sort contributes $O(n)$ total.

Cases: Worst: $O(n \log n)$. Average: $O(n \log n)$. Best: $O(n \log n)$.

Algorithm 6: Introsort (based on C++ implementation introsort.cpp)

Data: Array V (1-based index), start s , end e , depth $depth$

Result: Sorted segment $V[s..e]$

Procedure IntrosortRecursive($V, s, e, depth$):

```

     $\text{size} \leftarrow e - s + 1$ ;
    if  $\text{size} \leq 1$  then
        | return
    end
    ;
    if  $depth < \lfloor \log_2(\text{size}(V) - 1) \rfloor$  then
        | if  $\text{size} < 16$  then
        | | InsertionSortSegment ( $V, s, e$ )
        | end
        ;
        | else
        | |  $\text{pivot\_idx} \leftarrow \text{Partition}(V, s, e, V[s])$ ;
        | | IntrosortRecursive
        | | ( $V, s, \text{pivot\_idx} - 1, depth + 1$ );
        | | IntrosortRecursive
        | | ( $V, \text{pivot\_idx} + 1, e, depth + 1$ );
        | end
    end
    ;
    end
    else
        | if  $\text{size} < 16$  then
        | | InsertionSortSegment ( $V, s, e$ )
        | end
        ;
        | else
        | | HeapSortSegment ( $V, s, e$ )
        | end
    end
    ;
    end
    ;
```

In Conclusion: Introsort (C++ version): **Unstable, not strictly in-place** (stack). Advantage: Robust $O(n \log n)$ worst-case with $O(n \log n)$ average speed. Disadvantage: Complex, unstable, sub-optimal pivot choice (differs from more robust versions).

4 EXPERIMENTAL RESULTS AND ANALYSIS

This section presents the empirical performance evaluation of the implemented sorting algorithms based on execution time and memory usage, referencing the figures provided. Experiments were conducted on a system running **Ubuntu 22.04 LTS** with code compiled using **g++ 11.4.0**. The input datasets included **ascending sorted, descending sorted (covering reverse sorted), random, and partially sorted data**, as required. Input sizes spanned from **1,000 (1K) to 1,000,000 (1M) elements**. All implemented algorithms were verified for **correctness (sorting accuracy)**. Memory usage reported represents the **average peak resident set size (RSS)** across runs, measured using appropriate system tools. The source

code and datasets used for these experiments are publicly available at [here](#)[7].

Implementation Notes impacting results:

- Quick Sort utilized a queue-based approach instead of recursion for inputs exceeding 10K elements due to observed heap-buffer-overflow in worst-case scenarios with deep recursion. The memory plot reflects lower average usage but doesn't represent the theoretical worst-case peak accurately compared to a purely recursive implementation that might crash. However, since no other sorting algorithms had crashed, it is reasonable to assume that the original Quick Sort's worst case scenario will use the biggest memory space.
- The memory comparison graph (Figure 1) averages peak memory usage across different input data types for each size, assuming the peak usage doesn't vary drastically with input type for most algorithms.
- Each test configuration (algorithm, data type, size) was executed a **minimum of 10 times**, and the average execution time and average peak memory usage were recorded to ensure reliability of results, showing low variance in timings.
- No algorithms used optimization techniques beyond their standard definitions except when a crucial part of the algorithm had to be replaced to avoid error (like the Quick Sort queue).
- The implemented Bubble Sort excluded early termination optimization intentionally for baseline $O(n^2)$ comparison against Selection/Insertion.
- The implemented Cocktail Shaker Sort also lacked early termination, similar to Bubble Sort, to observe its baseline behavior and compare with Bubble Sort.
- Quick Sort and Introsort used the first element as the pivot. While suboptimal and leading to $O(n^2)$ performance on sorted/reverse-sorted data, this choice was maintained to clearly demonstrate the impact of pivot selection and the effectiveness of Introsort's fallback mechanism.
- Due to extreme execution times for $O(n^2)$ algorithms, tests for the 1M element dataset were conducted only **5 times** for Bubble Sort, Cocktail Shaker Sort, Insertion Sort, and Library Sort, as obtaining 10 runs was computationally prohibitive while still providing a reasonable average.

Analysis based on Plots (Figures 1-6):

Memory Usage (Figure 1): This log-log plot clearly visualizes the auxiliary space complexity.

- **Unique Feature (Grouping and Time-Memory Correlation):** Figure 1 distinctly groups algorithms by auxiliary space complexity. The in-place algorithms (Heap, Selection, Insertion, Bubble, Comb, Cocktail) exhibit relatively flat lines, indicating $O(1)$ or slowly growing auxiliary space, while not-in-place algorithms (Merge, Library, Tournament, Quick) show lines with steeper slopes, reflecting their $O(n)$ or $O(\log n)$ average / $O(n)$ worst-case space requirements. By examining the log-log time vs. memory scatter plot (Figure 2), a slight positive correlation between average execution time and memory usage is observed. While suggesting memory usage isn't perfectly constant even for "in-place"

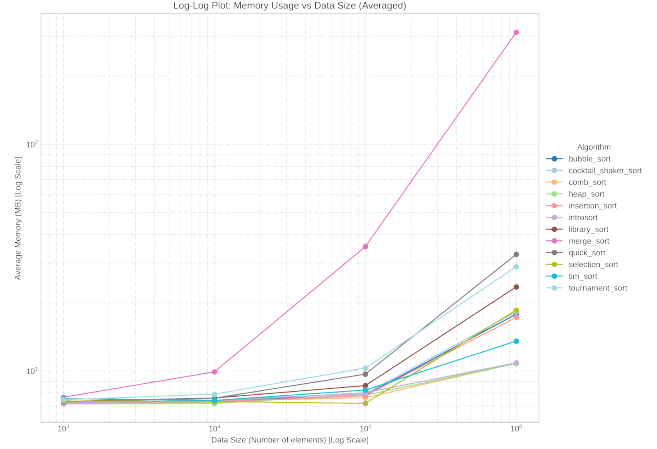


Figure 1: Execution Memory Comparison on Data (Log-Log Scale)

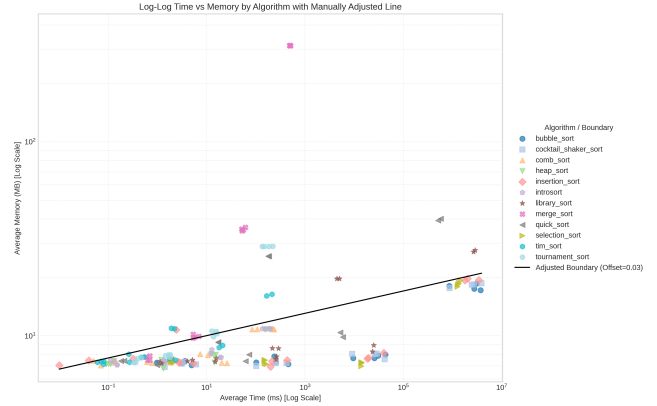


Figure 2: Execution Time vs. Memory Scatter Plot (Log-Log Scale)

sorts (due to measurement overhead or system factors), the fundamental grouping holds: in-place algorithms occupy a lower memory band compared to those requiring significant auxiliary structures.

Ascending Data (Figure 3): Usually highlights best-case performance and adaptivity.

- **Unique Feature (Adaptivity):** Insertion Sort and Tim Sort approach linear $O(n)$ time, vastly outperforming others due to minimal work on sorted data. Tim Sort's initial run detection and merging overhead make it slightly slower than pure Insertion Sort for already sorted data, but still effectively linear.
- **Consistent Algorithms:** Merge, Heap, Tournament, Intro, Comb maintain around $O(n \log n)$ performance, showing less sensitivity to this input type.
- **Unique Feature (Non-Optimized $O(n^2)$):** Unoptimized Bubble and Cocktail Shaker Sorts (without early exit flags)

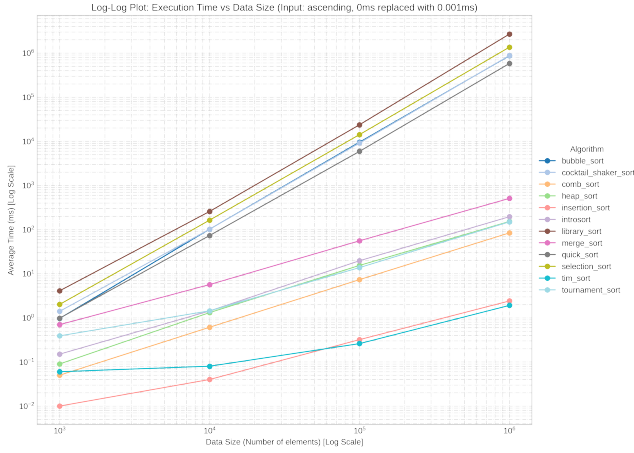


Figure 3: Execution Time Comparison on Ascending Sorted Data (Log-Log Scale)

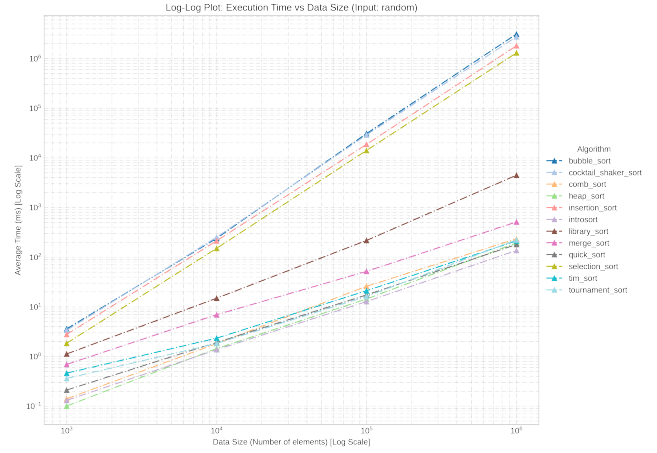


Figure 5: Execution Time Comparison on Random Data (Log-Log Scale)

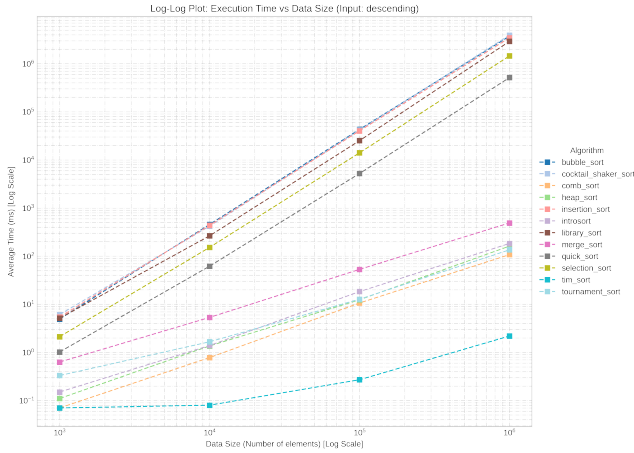


Figure 4: Execution Time Comparison on Descending Sorted Data (Log-Log Scale)

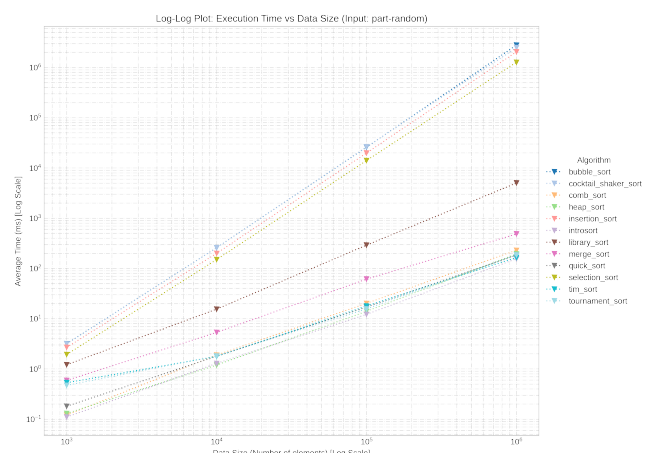


Figure 6: Execution Time Comparison on Part-random Data (Log-Log Scale)

remain $O(n^2)$, showing identical poor performance. If optimized, they would achieve $O(n)$. Quick Sort, with its first-element pivot, degenerates to $O(n^2)$, similar to Selection Sort. Library Sort also shows $O(n^2)$ behavior, potentially exacerbated by minimal gap usage and maximal shifting on sorted input.

Descending Data (Figure 4): Usually triggers worst-case behavior for simple algorithms.

- **Unique Feature (Tim Sort Adaptivity):** Despite being the reverse of sorted order, Tim Sort effectively detects the descending run, reverses it in $O(n)$ time, and proceeds efficiently, maintaining near $O(n)$ performance, demonstrating its robustness.
- **Consistent Algorithms:** Merge, Heap, Tournament, Intro, Comb remain $O(n \log n)$. Introsort avoids the $O(n^2)$ Quick Sort trap by switching to Heap Sort based on recursion depth.

Comb Sort's large initial gaps help avoid the $O(n^2)$ behavior seen in Bubble Sort.

- **Unique Feature ($O(n^2)$ Worst Cases):** Insertion Sort requires maximum shifts. Unoptimized Bubble/Cocktail perform maximum swaps/comparisons. Selection Sort's comparison count is unaffected. First-pivot Quick Sort again suffers from maximally unbalanced partitions. Library Sort likely faces worst-case shifting scenarios. All clearly degrade to $O(n^2)$.

Random Data (Figure 5): Shows average-case performance.

- **Unique Feature (Clear Separation):** A distinct performance gap separates the $O(n \log n)$ algorithms from the $O(n^2)$ algorithms, clearly demonstrating the asymptotic difference in scalability. No algorithms exhibit $O(n)$ average-case behavior here.

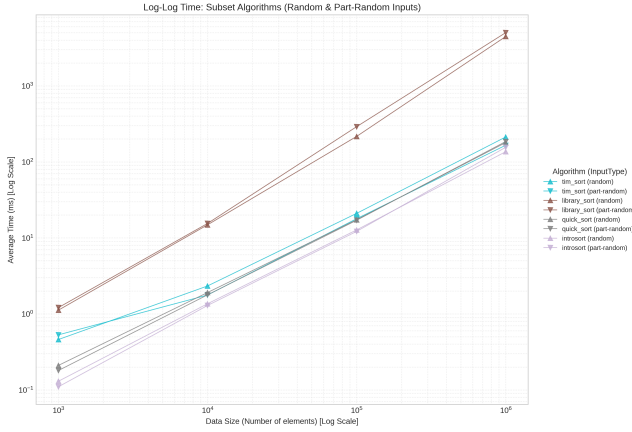


Figure 7: Execution Time Comparison with Random and Part-random Data (Log-Log Scale)

- **Unique Feature (Top Performers):** Quick Sort (average case), Introsort, Tim Sort, Heap Sort, Tournament Sort, and Comb Sort form the leading group, all exhibiting strong $O(n \log n)$ performance.
- **Unique Feature (Mid-Tier $O(n \log n)$):** Merge Sort and Library Sort, while still $O(n \log n)$ on average, show slightly higher constant factors or overhead compared to the top performers in this implementation, placing them in a slightly slower tier but vastly better than $O(n^2)$.
- **Unique Feature (Slowest Group):** Bubble, Cocktail Shaker, Insertion, and Selection Sort clearly cluster together as the slowest $O(n^2)$ group, impractical for large random datasets.

Partially Sorted Data (Figure 6, 7): Evaluates adaptivity and performance on near-sorted inputs.

- **Unique Feature (General Trend):** Most algorithms show performance characteristics similar to the random data case, suggesting the level of "partial sortedness" wasn't sufficient to trigger significant best-case behavior except for highly adaptive sorts.
- **Comparison with Random (Tim, Library, Intro, Quick):** As seen in Figure 7, Tim Sort leverages the existing order, showing a noticeable performance improvement compared to random data, highlighting its adaptivity. Conversely, Library Sort's performance slightly degrades, possibly because nearby elements cluster, reducing effective gap usage and increasing linear shifting costs during insertion. Quick Sort and Introsort also show a slight performance decline compared to random, likely due to the increased probability of encountering somewhat unbalanced partitions even before hitting the worst-case depth limit.

Crucially, all implemented algorithms produced correctly sorted output across all tested datasets and sizes, confirming their functional correctness alongside the performance characteristics observed.

Performance Validation and Implementation Impact. The experimental results strongly validate the theoretical complexities,

highlighting the superior scalability of $O(n \log n)$ algorithms over $O(n^2)$ counterparts for large datasets. The consistency of timings across multiple runs (low standard deviation) lends confidence to the measured averages. Algorithms sharing similar core mechanics performed comparably within their complexity class (e.g., unoptimized Bubble/Cocktail, Heap/Tournament). However, implementation details proved critical: Comb Sort's gap strategy significantly outperformed baseline Bubble Sort, while suboptimal choices like the first-element pivot in Quick Sort/Introsort or omitting early termination in Bubble/Cocktail clearly demonstrated their theoretical weaknesses on specific inputs. These choices emphasize the importance of careful design and optimization for practical performance. While not directly measured, variations hint at cache effects (e.g., Merge vs. Quick). This study's limitations include focusing solely on integer data and not directly quantifying cache behavior or instruction counts.

Algorithm Characteristics and Practical Recommendations.

The results underscore the significant advantages of adaptivity (Tim Sort, Insertion Sort exploiting existing order for near-linear time) and hybrid strategies (Introsort, Tim Sort combining strengths for robustness). While pure Insertion Sort excels on small or nearly sorted inputs, Tim Sort's balanced approach offers superior worst-case guarantees and excellent general performance. Introsort provides strong average speed with worst-case protection. These make Tim Sort (especially if stability is needed) and Introsort compelling choices for general-purpose libraries. The choice among algorithms involves trade-offs: $O(n \log n)$ average time is crucial for large n , stability is needed for multi-key sorts, in-place operation saves memory, and adaptivity boosts performance on non-random data. **Ultimately, this comparative study highlights the critical interplay between theoretical algorithm design and practical performance implications across diverse scenarios.**

References

- [1] 2023. TimSort – Data Structures and Algorithms Tutorials. GeeksforGeeks. <https://www.geeksforgeeks.org/timsort/> Last updated: 2023-11-20. Accessed: 2025-04-15.
- [2] Michael Ashley Bender, Martin Farach-Colton, and Miguel Angel Mosteiro. 2006. Insertion Sort is $O(n \log n)$. *Theory of Computing Systems* 39, 3 (2006), 391–397. doi:10.1007/s00224-005-1237-z
- [3] Włodzimierz Dobosiewicz. 1980. An efficient variation of bubble sort. *Inf. Process. Lett.* 11, 1 (1980), 5–6. doi:10.1016/0020-0190(80)90022-8
- [4] Charles Antony Richard Hoare. 1961. Algorithm 64: Quicksort. *Commun. ACM* 4, 7 (1961), 321. doi:10.1145/366622.366644
- [5] Donald Ervin Knuth. 1998. *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional, Reading, MA.
- [6] David Rea Musser. 1997. Introspective sorting and selection algorithms. *Software: Practice and Experience* 27, 8 (1997), 983–993. doi:10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-1
- [7] skytre0. 2025. CSE33101: Git Repository. GitHub Repository. <https://github.com/skytre0/CSE33101.git> Accessed: 2025-04-15.
- [8] Alexander Stepanov and Aaron Kershenbaum. 1986. *Using tournament trees to sort*. CATT Technical Report CATT-86-13. Polytechnic Institute of New York, Brooklyn, NY.